



DOCUMENTACIÓN PROYECTO CHURCH POS (SISTEMA DE PUNTO DE VENTA POR EVENTOS)

Contenido

Objetivo.....	5
Actores.....	5
1. El cajero (admin / usuario)	5
2. El despacho (cocina / entrega)	5
Requerimientos funcionales.....	5
• REF-001 (Gestión de eventos)	5
• RF-002 (Menú dinámico).....	5
• RF-003 (Ciclo de pedido)	5
▪ Estados:	5
• RF-004 (Visualization KDS – Kitchen Display System)	5
• RF-005 (Cierre y exportación)	6
• RF-006 (Billetera temporal)	6
Requerimientos no funcionales	6
• RNF-001 (Disponibilidad offline)	6
• RNF-002 (Arquitectura centralizada)	6
• RNF-003 (Latencia).....	6
Lógica de Negocio.....	6
A. Sesión de venta (Event – Driven Domain)	6
B. Comunicación en tiempo real (Pantalla de cocina)	7
C. Estructura de datos para Excel	7
Diagrama Conceptual	8
Especificación del módulo de gestión de saldos (Billetera).....	9
Tabla: clientes.....	9
Tabla: movimiento de saldos	10
Diagrama de Flujo: procesos de venta y gestión de saldo	11
1. Venta convencional	11
2. Generación de Billetera (Onboarding).....	11
3. Consumo con saldo Preexistente	11

Diagrama de Clases UML (modelo del dominio)	13
A. Encapsulamiento y tipos de datos.....	13
▪ Money Pattern (BigDecimal)	13
▪ Enums (estadoOrden, TipoMovimiento)	13
B. Relaciones estructurales	13
1. Evento como “Root Aggregate”	13
2. Composición fuerte (Orden – Detalle)	13
3. Relación opcional (Cliente – Orden).....	13
Diagrama de Despliegue	14
Diagrama de despliegue de infraestructura local	14
▪ Nodo servidor	14
▪ Nodos cliente	14
Documentación de Endpoints	15
Modulo A: Gestión del evento (Jornada).....	15
Módulo B: Menú y productos.....	15
Módulo C: Gestión de clientes y billetera	16
Módulo D: Ventas y cocina	16
Contrato de Interfaz (API contact)	17
Estructura proyecto GitHub	17
1. La estructura Raíz (The Big picture)	17
2. Backend – Estructura hexagonal simplificada	18
3. Frontend (React + Typescript).....	18
4. Carpeta “docs”	19
Guía de Instalación y Ejecución (Developer Runbook).....	19
Pre requisitos del Sistema (Host).....	19
• Sistema Operativo	19
• Java Runtime	19
• Node.js.....	19
• Base de datos	19

Variables de entorno (.env / application.properties)	20
Comandos de arranque (Cheatsheet).....	20
Roadmap y mejoras futuras	21
Fase 1 (MVP - Actual)	21
Fase 2 (Mejora UX)	21
Fase 3 (seguridad)	21
Fase 4 (Escalabilidad).....	21

Objetivo

Gestionar ventas de comida rápidas en eventos específicos, con menús efímeros y flujo de despacho en tiempo real.

Actores

1. El cajero (admin / usuario)

Configura el menú del día, toma los pedidos y cobra.

2. El despacho (cocina / entrega)

Ve que pedidos están pendientes y marca cuando se entregaron.

Requerimientos funcionales

- REF-001 (Gestión de eventos)

El sistema debe permitir crear una “Jornada de venta” (ej. “Venta domingo 12”). Cada jornada es independiente.

- RF-002 (Menú dinámico)

El menú pertenece a la jornada. Un plato “Tacos” en la jornada 1 puede costar diferente en la jornada 2. No hay “menú global” fijo que se arrastra siempre, o si lo hay, se clona para el evento.

- RF-003 (Ciclo de pedido)

El pedido tiene estados. No es solo “Comprado”.

- Estados:

1. Pagado
2. Pendiente_entrega
3. Entregado

- RF-004 (Visualization KDS – Kitchen Display System)

Pantalla en tiempo real que filtra solo los pedidos en estado “pendiente_entrega”

- RF-005 (Cierre y exportación)

Generación de reporte en formato .xlsx (Excel) compatible con Microsoft Office, agrupando ventas por jornada.

- RF-006 (Billetera temporal)

El sistema debe permitir registrar depósitos de efectivo a un cliente temporal y descontar el saldo en cada compra, así como permitir el retiro del sobrante al final.

Requerimientos no funcionales

- RNF-001 (Disponibilidad offline)

El sistema debe ser capaz de operar en una red de área local (LAN) aislada, sin dependencia de conectividad a internet (WAN).

- RNF-002 (Arquitectura centralizada)

El servidor (Backend + base de datos) residirá en una única instancia física (On premise) que actuará como Host para los clientes móviles.

- RNF-003 (Latencia)

La comunicación entre clientes y servidor debe ser en tiempo real (< 200 ms) mediante el protocolo WebSocket para la actualización de comandas en comida.

Lógica de Negocio

A. Sesión de venta (Event – Driven Domain)

En un restaurante normal, el menú es estático. Pero en este proyecto, el menú cambia con cada venta según el grupo de personas que realice la venta.

- Se implementará una entidad **Evento** o **Jornada**.
- Los **productos** se asocian a la **jornada**. Esto permite que el historial de ventas pasadas no se rompa si se borra un plato del menú de “hoy”.

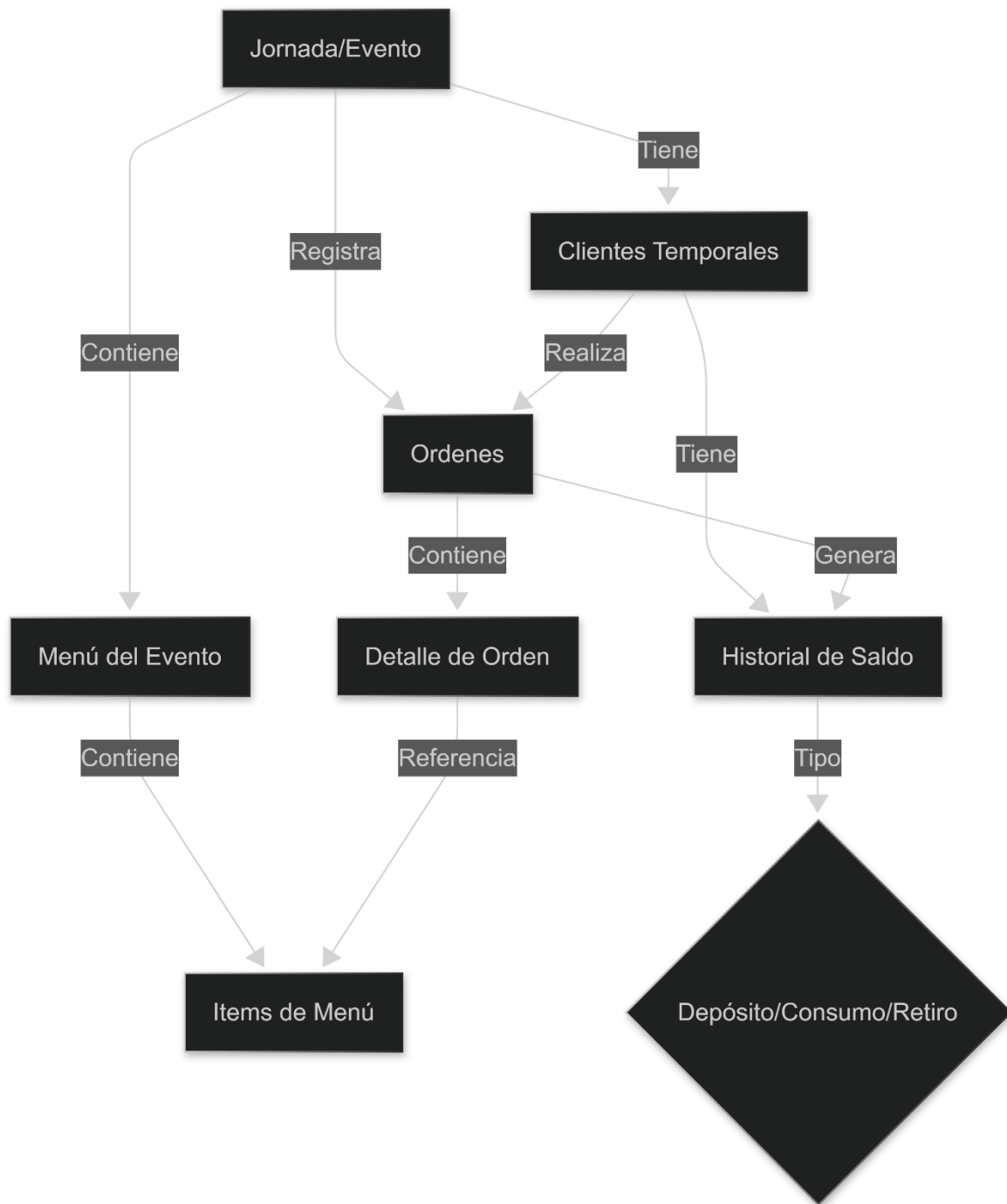
B. Comunicación en tiempo real (Pantalla de cocina)

Para que la pantalla de pedidos se actualice automáticamente sin necesidad de que se presione un botón, se utilizará **WebSockets** o **Server-Sent Events (SSE)**. Cuando la caja guarda el pedido el Backend “empuja” la notificación a la pantalla de la cocina instantáneamente. Se usará Spring Boot.

C. Estructura de datos para Excel

Se usará la librería **Apache POI** en Java. Este es el estándar industrial para general Excels reales, no solamente CSVs, permitiendo formatos y fórmulas si así fuera necesario.

Diagrama Conceptual



La entidad **Historial de Saldo** actúa como un punto de convergencia transaccional. Se vincula con **clientes temporales** para identificar la propiedad de fondos, y con **ordenes** para mantener la trazabilidad (adit trial) de los consumos realizados, justificando así cada deducción de saldo.

Especificación del módulo de gestión de saldos (Billetera)

Este módulo gestiona la persistencia de cuentas corrientes temporales para agilizar el cobro en caja, eliminando la necesidad de transacciones en efectivo por cada ítem adquirido.

Tabla: clientes

Entidad que representa al portador de un saldo temporal durante la vigencia de un evento. Actúa como una cuenta contable transitoria.

Campo	Tipo de Dato (SQL)	Restricciones	Descripción Técnica
id	BIGINT	PK, AI	Identificador único del cliente.
nombre	VARCHAR(100)	NOT NULL	Nombre o alias para identificar al cliente en caja (ej: "Hna. Marta").
saldo_actual	DECIMAL(10,2)	DEFAULT 0	Campo de Caché. Almacena el saldo disponible en tiempo real. Se actualiza mediante triggers o lógica de negocio tras cada insert en movimientos_saldo.
evento_id	BIGINT	FK	Vincula al cliente con una jornada específica. Garantiza que los saldos no persistan entre eventos distintos.

Tabla: movimiento de saldos

Tabla de auditoría inmutable (Ledger). Registra cada alteración matemática del saldo del cliente. Es la “fuente de la verdad” financiera del sistema.

Campo	Tipo de Dato (SQL)	Restricciones	Descripción Técnica
id	BIGINT	PK, AI	Identificador único de la transacción.
cliente_id	BIGINT	FK	Referencia al propietario del saldo afectado.
orden_id	BIGINT	FK, NULLABLE	Referencia a la venta que originó el débito. Es NULL si el movimiento es un Depósito o Retiro manual.
tipo	ENUM	NOT NULL	Valores permitidos: 'DEPOSITO', 'CONSUMO', 'RETIRO'. Define la naturaleza contable de la operación.
monto	DECIMAL(10,2)	NOT NULL	Valor absoluto de la transacción.
fecha_hora	DATETIME	DEFAULT NOW()	Estampa de tiempo exacta para auditoría y arqueo de caja.

Diagrama de Flujo: procesos de venta y gestión de saldo

El siguiente diagrama detalla la lógica condicional que el sistema ejecuta al momento de finalizar una orden. Se destacan 3 rutas críticas:

1. Venta convencional

El cliente paga el monto exacto o recibe su cambio en efectivo. El flujo es lineal y cierra la transacción inmediatamente.

2. Generación de Billetera (Onboarding)

Se detecta un excedente en el pago (ej. Billete de Q100 para un consumo de Q15). El sistema intercepta el flujo normal y ofrece la opción de “retener cambio”. Si se acepta, se instancia un nuevo cliente y se ejecutan dos movimientos financieros atómicos: un **Depósito** por el total del billete entregado y un **Consumo** por el total de la orden, dejando el remanente como saldo a favor.

3. Consumo con saldo Preexistente

El sistema valida la solvencia del cliente temporal. Si el saldo es insuficiente, se solicita únicamente el diferencial (Gap), el cual se ingresa como un depósito complementario antes de procesar el consumo total.

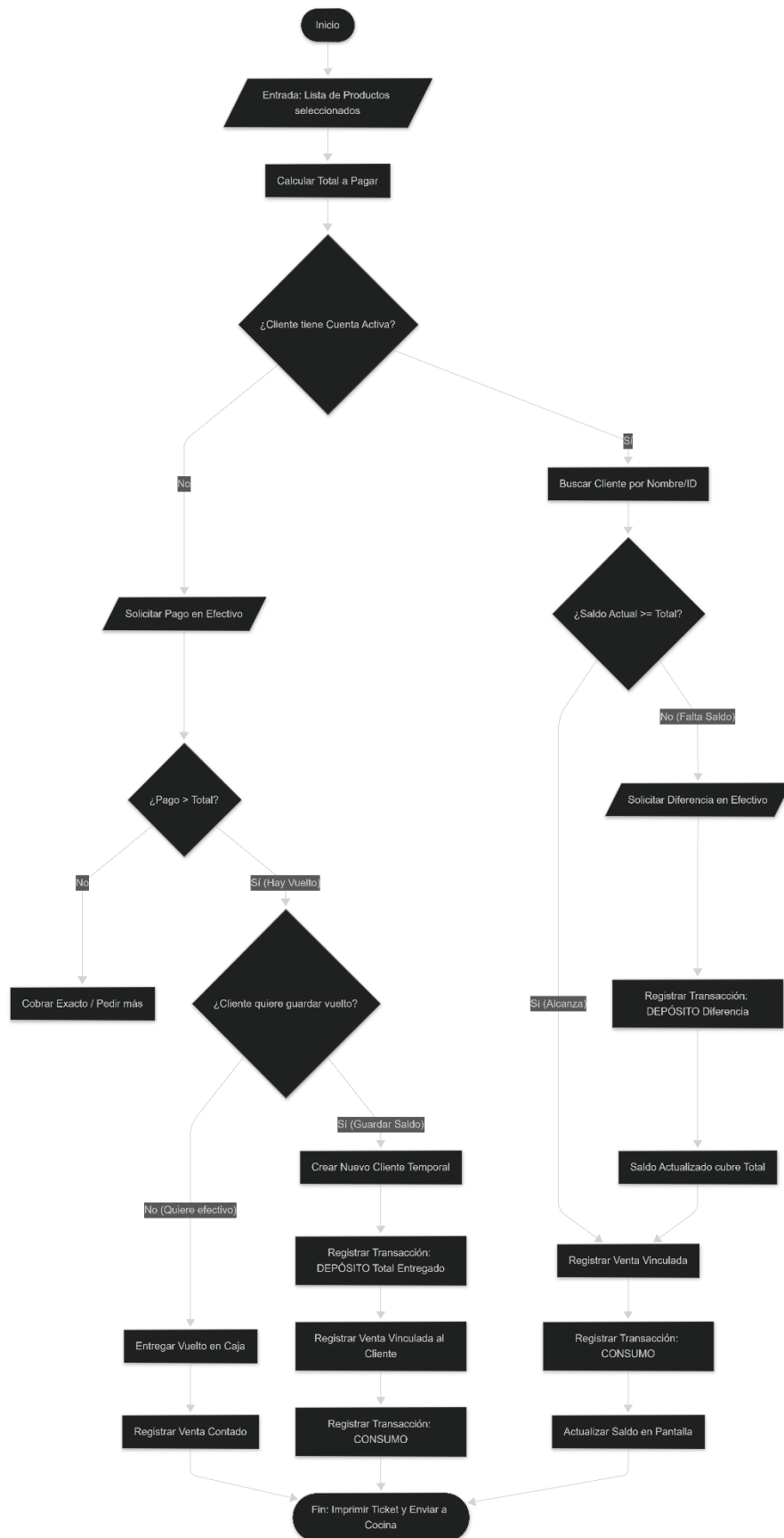


Diagrama de Clases UML (modelo del dominio)

A. Encapsulamiento y tipos de datos

- Money Pattern (BigDecimal)

Para todos los campos de dinero se utiliza BigDecimal en lugar de Double o Float. Esto es porque en sistemas financieros, los tipos flotantes generan errores de redondeo. BigDecimal es exacto y es el estándar industrial en Java.

- Enums (estadoOrden, TipoMovimiento)

No usamos Strings sueltos (“Pagado”, “Debe”). Usamos enumeraciones para garantizar que el código sea seguro y evitar errores de dedo (Typos).

B. Relaciones estructurales

1. Evento como “Root Aggregate”

La clase **evento** es la raíz. Todo nace ahí. Un evento tiene productos y tiene órdenes. Si se borra el evento (lógicamente), todo lo demás pierde contexto.

2. Composición fuerte (Orden – Detalle)

- El rombo negro entre Orden y DetalleOrden significa Composición.
- Lógica: un detalle de orden (ej. 3 tacos) no puede existir sin su orden padre. Si la orden muere el detalle también.

3. Relación opcional (Cliente – Orden)

Una orden puede tener un cliente (si pagó con saldo) o puede no tenerlo (si pagó cash anónimo)

Diagrama de Despliegue

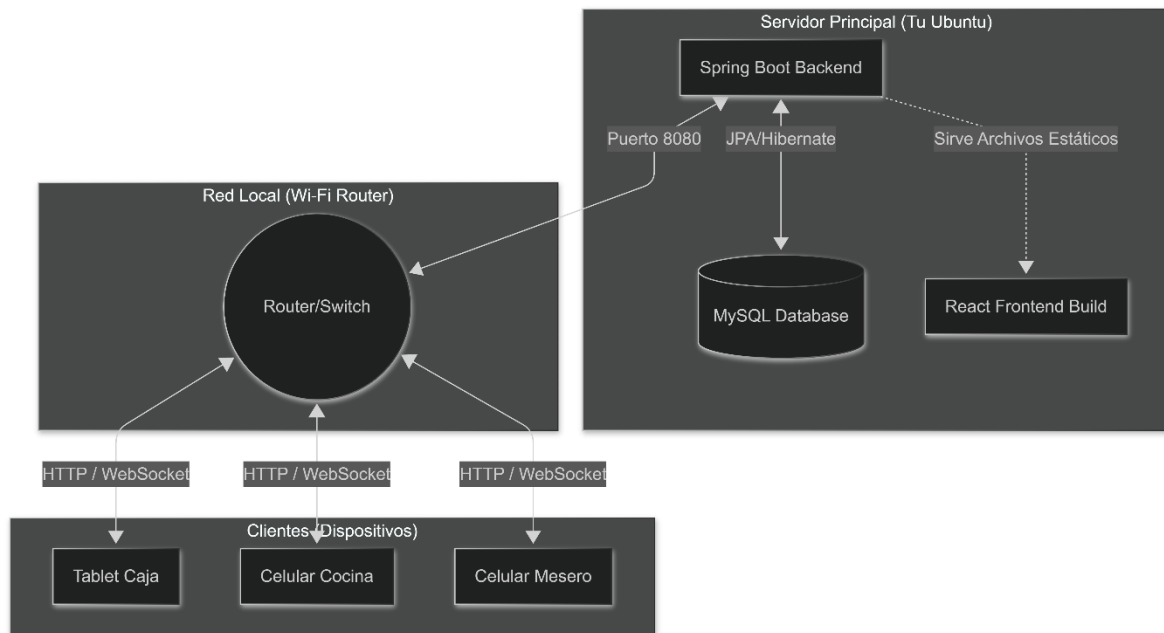


Diagrama de despliegue de infraestructura local

Este diagrama ilustra la arquitectura física del sistema. Se utiliza como un modelo cliente – servidor sobre una red TCP/IP local.

- **Nodo servidor**

Una estación de trabajo Ubuntu aloja el motor de base de datos MySQL y el contenedor de aplicaciones Java (Spring Boot Apache Tomcat embebido).

- **Nodos cliente**

Dispositivos móviles genéricos acceden a una aplicación a través de un navegador web estándar, comunicándose con el servidor mediante peticiones HTTP REST y canales bidireccionales WebSocket a través del router local.

Documentación de Endpoints

Modulo A: Gestión del evento (Jornada)

Para configurar el día de venta

Método	Endpoint (URL Relativa)	Descripción Funcional
POST	/api/v1/eventos	Abre una nueva jornada de ventas (ej: "Venta Domingo 12").
GET	/api/v1/eventos/activo	Obtiene los datos del evento que está ocurriendo ahora mismo .
PUT	/api/v1/eventos/{id}/cerrar	Cierra la caja y genera el reporte final del día.

Módulo B: Menú y productos

Para que el cajero sepa que vender

Método	Endpoint	Descripción Funcional
GET	/api/v1/productos/menu	Obtiene la lista de platos disponibles con sus precios actuales.
POST	/api/v1/productos	Agrega un nuevo plato al menú (ej: se acabaron los tacos, ahora vendemos tostadas).

Módulo C: Gestión de clientes y billetera

Método	Endpoint	Descripción Funcional
GET	/api/v1/clientes/buscar?q=nombre	Busca si "Hermana Marta" ya tiene cuenta abierta hoy.
POST	/api/v1/clientes	Registra un cliente nuevo y deposita su saldo inicial (Onboarding).
GET	/api/v1/clientes/{id}/saldo	Consulta cuánto dinero le queda a un cliente específico.

Módulo D: Ventas y cocina

Método	Endpoint	Descripción Funcional
POST	/api/v1/ordenes	Crear Venta. Recibe la lista de platos y, si aplica, el ID del cliente para descontar saldo.
GET	/api/v1/ordenes/pendientes	Pantalla de Cocina. Devuelve solo las órdenes que no se han entregado.
PUT	/api/v1/ordenes/{id}/entregar	Acción de Cocina. Marca la orden como lista y la quita de la pantalla.
GET	/api/v1/ordenes/reporte	Descarga el Excel con el total vendido (Solo Admin).

Contrato de Interfaz (API contract)

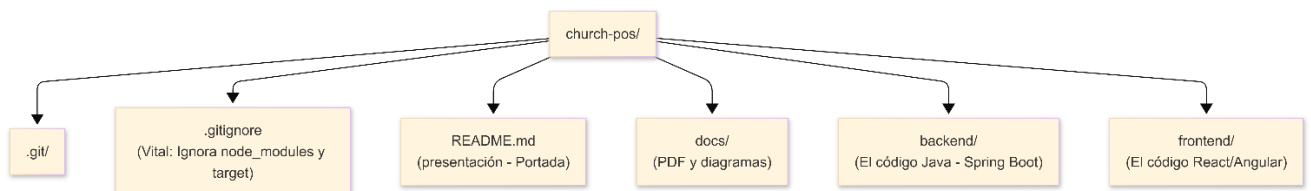
POST /api/v1/ordenes - Ejemplo de Payload

```
{
  "clienteId": 45,          // (Opcional) ID si paga con saldo. Null si es efectivo
                             exacto.
  "eventoId": 1,           // ID de la jornada actual
  "detalles": [
    {
      "productoId": 101, // ID de Tacos
      "cantidad": 2,
      "precioUnitario": 15.00
    },
    {
      "productoId": 102, // ID de Soda
      "cantidad": 1,
      "precioUnitario": 5.00
    }
  ],
  "total": 35.00,
  "metodoPago": "SALDO" // 0 "EFECTIVO"
}
```

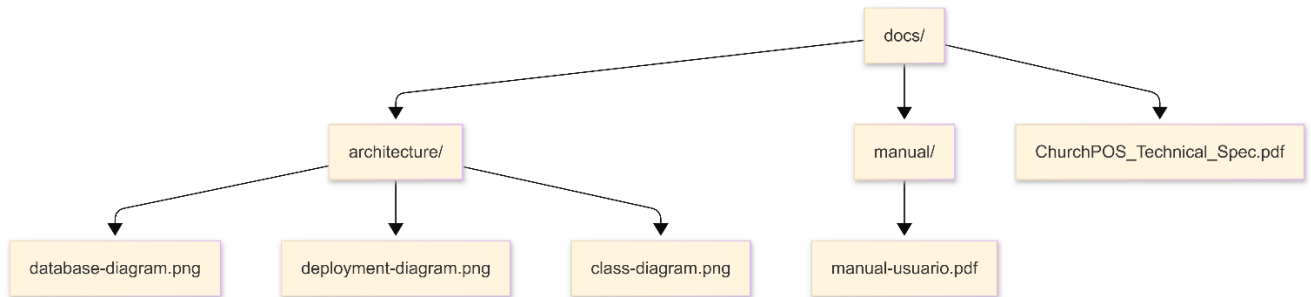
Estructura proyecto GitHub

1. La estructura Raíz (The Big picture)

Un solo repositorio llamado church-pos



4. Carpeta “docs”



Guía de Instalación y Ejecución (Developer Runbook)

Pre requisitos del Sistema (Host)

- **Sistema Operativo**
Linux (Ubuntu 22.04 LTS recomendado) / Windows 10+
- **Java Runtime**
JDK 17 o superior.
- **Node.js**
Versión 18.x o superior (para compilar el frontend)
- **Base de datos**
MySQL 8.0 corriendo en el puerto 3306

Variables de entorno (.env / application.properties)

Variable	Descripción	Valor por Defecto (Dev)
DB_URL	URL de conexión JDBC	jdbc:mysql://localhost:3306/church_pos
DB_USER	Usuario de base de datos	root
DB_PASS	Contraseña de base de datos	(Vacio) o admin
SERVER_PORT	Puerto del Backend	8080

Comandos de arranque (Cheatsheet)

BASH

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/church-pos.git

# 2. Levantar la Base de Datos (Si usas Docker, opcional)
docker-compose up -d mysql

# 3. Compilar y Ejecutar Backend
cd backend
./mvnw spring-boot:run

# 4. (En otra terminal) Levantar Frontend en modo desarrollo
cd frontend
npm install
npm run dev -- --host
```

Roadmap y mejoras futuras

Fase 1 (MVP - Actual)

Gestión de ventas en efectivo, saldos temporales y reportes simples en Excel.
Despliegue en red local.

Fase 2 (Mejora UX)

Implementación de impresión térmica directa (ESC/POS) desde el navegador para tickets físicos en cocina

Fase 3 (seguridad)

Implementación de roles (Admin vs Cajero) usando JWT (Jason Web Tokens) para evitar que un cajero modifique precios.

Fase 4 (Escalabilidad)

Migración a la nube (AWS / Render) para permitir pedidos online anticipados durante la semana.